

Progetto di Scalable and Cloud Programming

Matteo Galiazzo

Dipartimento di Informatica - Scienza e Ingegneria

Università di Bologna

`matteo.galiazzo@studio.unibo.it`

Anno Accademico 2025-26

Contents

1	Lavoro svolto	1
1.1	Pre-elaborazione dei dati	1
2	Approccio utilizzato	1
2.1	Gestione dei Dati e Strutture	2
2.2	Algoritmo di Analisi	2
2.3	Automazione e Cloud	2
3	Analisi di scalabilità e prestazioni	2
3.1	Risultati grezzi dei Benchmark	2
3.2	Setup Sperimentale	2
3.3	Risultati Sperimentali	3
3.3.1	Osservazioni preliminari	3
3.4	Impatto del Partizionamento	3
3.4.1	Sotto-utilizzo delle risorse con poche partizioni	3
3.4.2	Regime di overhead	3
3.5	Strong Scaling Analysis	4
3.5.1	Speedup	4
3.5.2	Strong Scaling Efficiency	4
3.6	Stima della Frazione Parallelizzabile (Legge di Amdahl)	4
3.7	Conclusioni	5

1 Lavoro svolto

L'obiettivo del progetto è realizzare un'applicazione distribuita per l'analisi di un dataset di eventi sismici, al fine di individuare la coppia di località caratterizzata dal massimo numero di co-occorrenze e di elencare le date in cui tali eventi si sono verificati.

Il codice sorgente completo e le istruzioni per l'esecuzione sono disponibili nella repository GitHub all'indirizzo: <https://github.com/gekoxyz/unibo-scalable-and-cloud>.

1.1 Pre-elaborazione dei dati

Al dataset sono state applicate le seguenti trasformazioni:

- Le coordinate geografiche (latitudine e longitudine) sono state approssimate alla prima cifra decimale con arrotondamento all'intero più vicino, per raggruppare gli eventi in macro-aree.
- Gli orari degli eventi sono stati rimossi, per considerare la co-occorrenza solo tramite la data.
- Sono stati rimossi gli eventi duplicati, per evitare che più eventi registrati nella stessa area e nello stesso giorno vengano conteggiati come co-occorrenze multiple.

2 Approccio utilizzato

L'implementazione segue un'architettura in due fasi progettata per separare il processing del dataset dall'analisi delle co-occorrenze.

2.1 Gestione dei Dati e Strutture

È stata definita la classe `Position` per rappresentare una cella geografica e fornire un ordinamento tra le coordinate. L'ordinamento consente di generare direttamente combinazioni non ordinate tra posizioni evitando normalizzazioni costose delle coppie (A, B) e (B, A) , riducendo il lavoro computazionale nella fase combinatoria.

Sia le posizioni sia le date sono state rappresentate tramite `Int`. Questa scelta riduce l'occupazione di memoria, migliora le prestazioni di serializzazione nello shuffle e rende più efficienti le operazioni di confronto e hashing rispetto all'uso di `String` o `Double`.

Il dataset è stato caricato dal bucket Google Cloud Storage e immediatamente ripartizionato tramite il metodo `repartition` per distribuire uniformemente i dati tra i worker ed evitare fenomeni di data skew (distribuzione non uniforme dei dati) nelle fasi successive, molto più costosi da correggere.

2.2 Algoritmo di Analisi

L'algoritmo dopo il pre-processing dei dati è strutturato nel modo seguente:

1. **Raggruppamento per data:** l'RDD viene aggregato per giorno utilizzando `aggregateByKey`, così da eseguire una combinazione locale nelle singole partizioni prima dello shuffle. Per l'aggregazione viene utilizzato un `HashSet[Position]`, che consente deduplicazione direttamente durante la fase di combinazione locale. In questo modo la quantità di dati trasferiti tra nodi dipende dal numero di posizioni distinte e non dal numero totale di eventi registrati. L'RDD risultante viene mantenuto in cache poiché riutilizzato in più azioni, evitando la ricostruzione dell'intero DAG di trasformazioni.
2. **Conteggio delle co-occorrenze:** per ciascuna data vengono generate $\binom{n}{2}$ combinazioni non ordinate tra le posizioni osservate. L'uso di posizioni precedentemente ordinate permette di evitare duplicazioni e di non dover normalizzare esplicitamente le coppie. Le frequenze vengono poi aggregate con `reduceByKey`, che effettua una riduzione locale prima dello shuffle mantenendo il traffico di rete limitato. La coppia con frequenza massima viene individuata tramite una `reduce` distribuita, senza raccogliere l'intero RDD sul processo centrale che coordina il job.
3. **Recupero delle date:** una volta identificata la coppia più frequente, si riutilizza l'RDD salvato precedentemente in cache per filtrare le sole date in cui entrambe le posizioni compaiono simultaneamente. Questo evita di ripetere l'intero processo di aggregazione, che rappresenta la parte più costosa della pipeline.

2.3 Automazione e Cloud

Per gestire l'esecuzione su Google Cloud DataProc, è stato sviluppato uno script Python (`run.py`) che automatizza il test delle performance. Gli step principali sono:

- Creazione del cluster con macchine `n2-standard-4` e dischi da 100GB.
- Sottomissione dei job Spark con le diverse configurazioni di partizionamento.
- Download dei risultati e eliminazione del cluster.

3 Analisi di scalabilità e prestazioni

3.1 Risultati grezzi dei Benchmark

La Tabella 1 riporta i tempi di esecuzione (in secondi) ottenuti variando il numero di worker del cluster e il numero di partizioni impostato nell'applicazione Spark. I test sono stati eseguiti su un cluster Google Cloud Dataproc con macchine di tipo `n2-standard-4`.

Workers	Partizioni						
	8	16	32	48	64	128	256
2	648.23	513.23	452.43	428.81	402.83	385.07	374.01
3	612.57	379.76	313.27	294.56	292.83	290.89	290.04
4	596.30	255.13	247.32	235.64	229.11	239.17	226.24

Table 1: Tempi di esecuzione (in secondi) al variare di worker e partizioni.

3.2 Setup Sperimentale

Gli esperimenti sono stati eseguiti su cluster Google Cloud Dataproc composti da:

- **Master:** 1 nodo `n2-standard-4`



Figure 1: Tempistiche per le diverse configurazioni di worker e partizioni

- **Worker:** 2, 3 o 4 nodi `n2-standard-4`
- **Storage:** 100GB per nodo

Ogni nodo `n2-standard-4` dispone di 4 vCPU. Il numero totale di core disponibili varia quindi da 8 a 16 al variare dei worker.

Per ciascuna configurazione del cluster sono stati testati diversi livelli di parallelismo impostando il numero di partizioni Spark a:

$$\{8, 16, 32, 48, 64, 128, 256\}$$

La dimensione del dataset è stata mantenuta costante, per testare lo **strong scaling**.

3.3 Risultati Sperimentali

La Tabella 1 riporta i tempi di esecuzione (in secondi) per tutte le configurazioni testate.

3.3.1 Osservazioni preliminari

1. L'aumento del numero di partizioni da 8 a 16 produce una riduzione significativa del tempo di esecuzione, in particolare con 3 e 4 worker.
2. Oltre le 64 partizioni, i miglioramenti diventano marginali.
3. L'aumento del numero di worker produce uno speedup significativo ma sub-lineare.

3.4 Impatto del Partizionamento

3.4.1 Sotto-utilizzo delle risorse con poche partizioni

Se abbiamo 8 partizioni possiamo avere al massimo 8 task concorrenti, e quindi nella configurazione con 4 worker, dei 16 core disponibili ne possiamo usare solo 8, con un utilizzo massimo teorico del 50%.

Questo spiega il tempo elevato osservato con 8 partizioni (596 s). L'incremento a 16 partizioni consente la saturazione dei core e produce un miglioramento drastico (255 s).

3.4.2 Regime di overhead

Per 4 worker si osserva che:

- 48 partizioni: 236 s
- 64 partizioni: 229 s
- 128 partizioni: 239 s

- 256 partizioni: 226 s

Oltre le 48 partizioni, i benefici diventano marginali. Questo suggerisce l'ingresso in un regime in cui l'overhead di scheduling, la gestione di task più piccoli e i costi di shuffle iniziano a compensare i benefici del bilanciamento del carico.

Il range 48-64 partizioni rappresenta quindi un buon compromesso tra granularità e overhead.

3.5 Strong Scaling Analysis

Poiché non è disponibile una configurazione a 1 worker, utilizziamo 2 worker come baseline.

3.5.1 Speedup

Lo speedup è definito come $S(n) = \frac{T(2)}{T(n)}$

Sostituendo i valori abbiamo:

- $S(2) = \frac{374}{374} = 1.00$
- $S(3) = \frac{374}{290} = 1.29$
- $S(4) = \frac{374}{226} = 1.65$

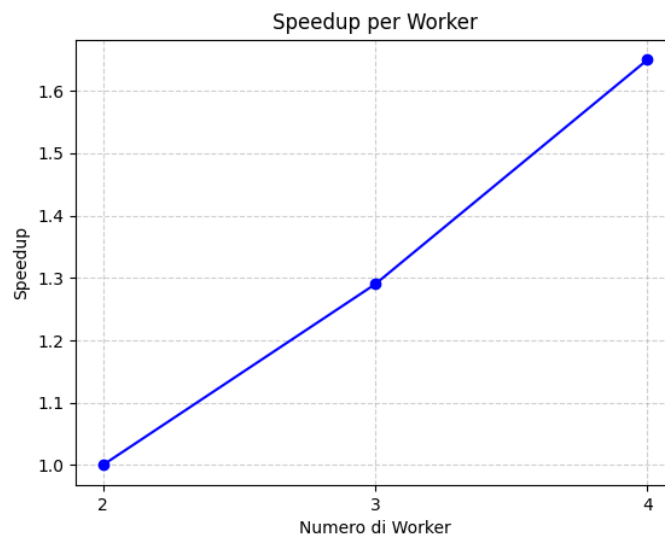


Figure 2: Speedup per Worker

3.5.2 Strong Scaling Efficiency

La strong scaling efficiency è definita come: $E(n) = \frac{2}{n} \cdot \frac{T(2)}{T(n)}$

Sostituendo i valori abbiamo:

- $E(2) = \frac{2}{2} \cdot \frac{374}{374} = 1.00$
- $E(3) = \frac{2}{3} \cdot \frac{374}{290} = 0.86$
- $E(4) = \frac{2}{4} \cdot \frac{374}{226} = 0.825$

3.6 Stima della Frazione Parallelizzabile (Legge di Amdahl)

Applicando la legge di Amdahl possiamo stimare la frazione parallelizzabile:

$$S(n) = \frac{1}{f + \frac{1-f}{P}}$$

dove f è la parte di lavoro che non può essere velocizzata, e P è il grado di parallelizzazione

- $P = 2$
- $S(n) = 1.65$

si ottiene: $f \approx 0.21$. Circa il **79% del workload risulta parallelizzabile**, mentre il restante 21% rappresenta frazione non parallelizzabile e costi dovuti a shuffle e sincronizzazione.

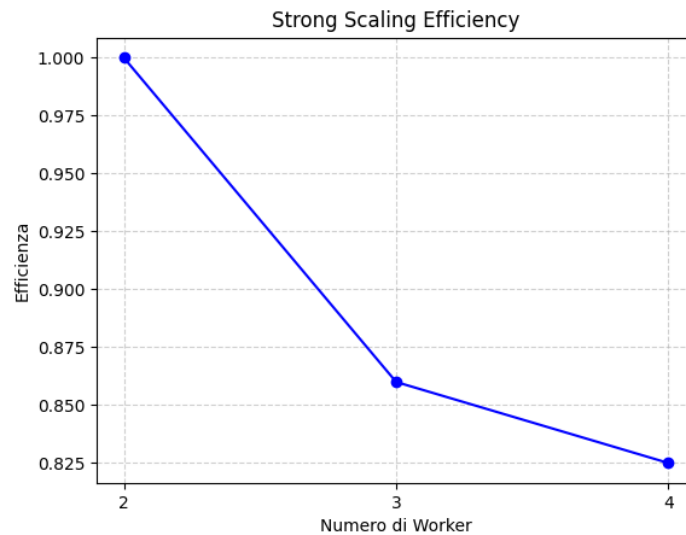


Figure 3: Strong Scaling Efficiency

3.7 Conclusioni

L'analisi mostra che:

1. Il parametro di partizionamento ha un impatto critico sulle prestazioni.
2. Oltre 48-64 partizioni (per 3-4 worker), i benefici si riducono a causa dell'overhead. Per 2 worker invece aumentare il numero di partizioni aiuta perchè così non c'è il rischio che qualche core rimanga a svolgere il lavoro di una partizione molto impegnativa mentre gli altri sono inattivi, ma il lavoro si bilancia meglio.
3. Lo strong scaling da 2 a 4 worker raggiunge:
 - Speedup = $1.65\times$
 - Efficienza = 82.5%
4. La frazione parallelizzabile stimata (79%) indica un'applicazione ben parallelizzata ma con una componente seriale non trascurabile.

Nel range testato il sistema mostra una buona scalabilità e non evidenzia ancora un limite. Ulteriori incrementi del numero di worker sarebbero necessari per identificare il punto di saturazione infrastrutturale.